

Third Party Platforms Integration Flow

This document outlines the login flow for Third-Party Platforms to integrate with mstock Trading API.

Overview

The login flow allows users to authenticate through Mstock's platform and then return to the vendor's application with authentication tokens and user details.

Note

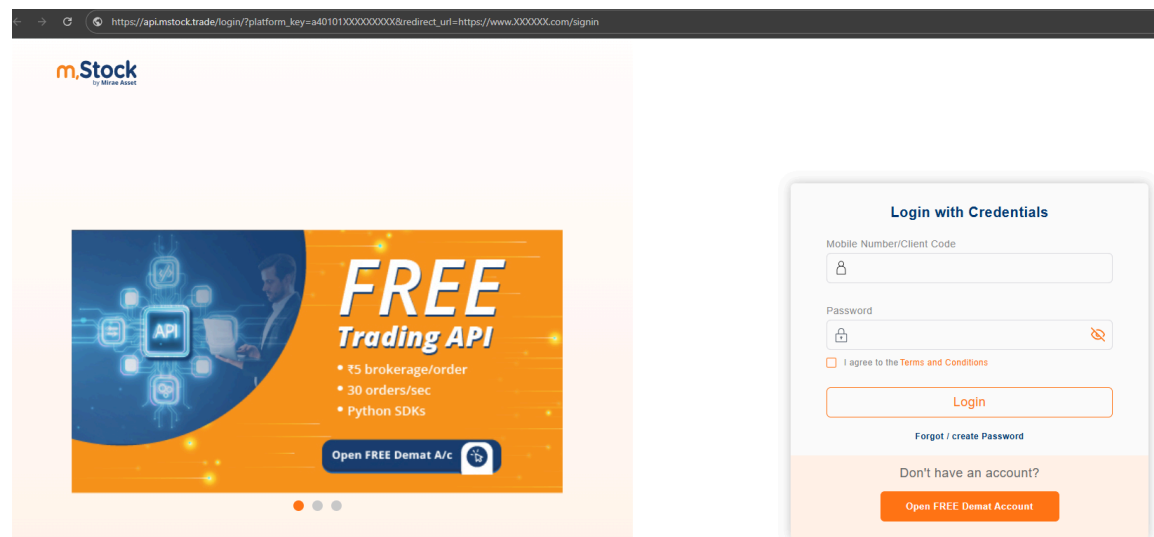
Users authenticate via Mstock's login page and are redirected back to your application with authentication tokens that enable your platform to API access.

Login Flow

Step 1: Redirect User to Mstock Login Page

Redirect your users to the Mstock login URL with your platform key and redirect URL:

```
https://api.mstock.trade/login/?  
platform_key=YOUR_PLATFORM_KEY&redirect_url=YOUR_REDIRECT_URL
```



Parameters:

- `platform_key`: Unique key configured by Mstock team for your application
- `redirect_url`: Your application URL where users will be redirected after successful login

Example:

```
https://api.mstock.trade/login/?  
platform_key=a40101XXXXXXXXXX&redirect_url=https://www.XXXXXXX.com/signin
```



Obtaining a Platform Key

To obtain your unique platform key, please reach out to our Partner Integration team at tradingapi@mstock.com with your company details and integration requirements. Our team will review your request and provide you with a dedicated platform key for your application.

Step 2: Handling of Authentication Data on your redirect url

After successful authentication, users will be redirected back to your redirect URL with the following parameters:

```
https://YOUR_REDIRECT_URL?  
Auth=JWT_TOKEN&ApiKey=ENCRYPTED_API_KEY(URI_ENCODED)&UserDetails=ENCRYPTED_USER_DETAILS(URI_ENCODED)
```

Response Parameters:

- **Auth** : JWT token containing user authentication information
- **ApiKey** : User-specific API key (unique for each user) - URI encoded
- **UserDetails** : Encrypted user information (client name, client code, exchange active segment) - URI encoded

Example:

```
https://www.abcd.com/signin?  
Auth=eyJhZ11NiIsxxxxxx&ApiKey=45f9qajh9W%2FAYUSOxxxxxxxxxxxxxv&UserDetails=kEwL519nxxxxxxxnUQ...
```

Step 3: Decrypt User Details

Use the following code to decrypt the URI encoded user details:

```
public static string DecryptString(string cipherText)  
{  
    try  
    {  
        byte[] iv = new byte[16];  
        byte[] buffer = Convert.FromBase64String(cipherText);  
  
        using (Aes aes = Aes.Create())  
        {  
            aes.Key = Encoding.UTF8.GetBytes(InstanceHelper.EncDecPrivateKey);  
            // "mstockqnt@2025pvtkey4forencdec25"  
            aes.IV = Encoding.UTF8.GetBytes(InstanceHelper.IVKey); //  
            "4512651296579784"  
            ICryptoTransform decryptor = aes.CreateDecryptor(aes.Key, aes.IV);  
  
            using (MemoryStream memoryStream = new MemoryStream(buffer))  
            {  
                using (CryptoStream cryptoStream = new
```

```
CryptoStream((Stream)memoryStream, decryptor, CryptoStreamMode.Read))
    {
        using (StreamReader streamReader = new
StreamReader((Stream)cryptoStream))
        {
            return streamReader.ReadToEnd();
        }
    }
}
}
}
catch (Exception ex)
{
    _logger.Error(ex);
    return null;
}
}
```

Decryption Keys: - Private Key: mstocXXXXkqnt@XXXXXXX - IV: XXXXXXXXXXXX



Obtaining a Private Key & IV

To obtain your unique private key & IV, please reach out to our Partner Integration team at tradingapi@mstock.com with your company details and integration requirements. Our team will review your request and provide you with a dedicated platform key for your application.

Step 4: Trading API Integration

Once authenticated, you can use the JWT token as access token to integrate with Mstock Trading APIs for various trading operations.

For detailed API documentation and endpoints, visit our [Trading API Documentation](#).

Implementation Notes

1. Store the Access Token securely for subsequent API calls
2. The API Key is unique for each user and should be included in API requests (remember to URI decode first)
3. Decrypt user details to get client information for personalization (URI decode before decryption)
4. Handle token expiration and implement refresh mechanisms as needed

Security Considerations

- Always use HTTPS for all API communications
- Store tokens securely and never expose them in client-side code
- Implement proper error handling for authentication failures
- Validate all incoming tokens before processing

**Warning**

Avoid API key exposure. It's unsafe to embed it in mobile apps or client code! Never let `access_token` be public.